



# Summer School 2025 Astronomy & Astrophysics

## Project Report

### Prepared By

Student Name: Rachit Saini

Institution Name: Thapar Institute of Engineering & Technology

Institution Roll No: 102304007

ISA Admission No: 695437

Project Name: Python in Astronomy: Tracking The International Space Station (ISS)

### Submitted To

Name: Mr. Anshuman Pathak

Designation: Program Supervisor

Institution: India Space Academy

# **CONTENTS**

- 1. Introduction**
- 2. Objective**
- 3. Methodology**
- 4. Code Overview**
- 5. Results and Graphical Output**
- 6. Conclusion**
- 7. References**

# INTRODUCTION

The International Space Station (ISS) is a habitable artificial satellite in low Earth orbit. Tracking its position and predicting its visibility from various locations on Earth is a fascinating application of orbital mechanics and data science. This project aims to provide a practical guide to tracking the ISS using Python, leveraging various libraries for geospatial data handling, orbital calculations, and visualization.

This report will detail the process of fetching real-time ISS Two-Line Element (TLE) data, calculating its pass times over specific locations, visualizing its ground track on a map, and exploring advanced analyses like orbital drift.

## OBJECTIVE

The primary objectives of this project are:

- To understand and apply concepts of geospatial data and orbital mechanics for satellite tracking.
- To gain proficiency in using key Python libraries such as skyfield, requests, numpy, and matplotlib.
- To develop a functional ISS tracker that can:
  - Fetch live TLE data for the ISS.
  - Calculate and display ISS pass times (rise, culmination, set) for a given observer location.
  - Visualize the ISS ground track on a world map.
  - Predict passes over multiple predefined locations.
  - Provide an interactive interface for real-time exploration.
- To analyze long-term orbital drift of the ISS, understanding the factors influencing its trajectory.
- To lay the groundwork for building a real-time web dashboard for ISS tracking.

## METHODOLOGY

The project will be implemented in a modular fashion, following the tasks outlined in the provided Jupyter Notebook. Each task builds upon the previous one, leading to a complete ISS tracking solution.

- 1. Environment Setup and Library Installation:** Ensure Python 3.8+ is installed and all necessary libraries (requests, skyfield, numpy, matplotlib, ipywidgets) are installed and imported.
- 2. Fetching ISS TLE Data:** Use the requests library to retrieve the latest TLE data for the ISS from a reliable source like CelesTrak. This data is crucial for accurate orbital calculations.
- 3. Calculating ISS Pass Times:** Utilize the skyfield library to parse the TLE data and calculate the precise rise, culmination (maximum elevation), and set times of the ISS for a user-specified latitude and longitude.
- 4. Visualizing the ISS Ground Track:** Employ matplotlib to plot the ISS's trajectory over a world map. This involves generating a series of ISS sub-point coordinates over a defined time window and overlaying them on a geographical projection.
- 5. Predicting Passes Over Multiple Locations:** Extend the pass time calculation to a list of predefined cities, automating the process and tabulating the results for easy comparison.
- 6. Interactive Exploration:** Integrate ipywidgets to create an interactive interface within a Jupyter environment, allowing users to dynamically select locations, adjust prediction windows, and trigger recalculations and re-plotting.
- 7. Advanced Analysis – Orbital Drift:** Investigate the long-term changes in ISS orbital elements (inclination, eccentricity, period) by analyzing historical TLE data and visualizing trends.
- 8. Building a Real-Time Dashboard Application (Conceptual):** Outline the steps for transforming the core functionality into a simple

web application using frameworks like Flask or Streamlit, including API endpoints and frontend visualization.

Throughout the methodology, emphasis will be placed on robust error handling, clear data validation, and producing human-readable outputs.

## CODE OVERVIEW

The following sections present the Python code for each task. Each code block is self-contained and includes comments explaining the logic and functionality.

### Prerequisites and Task 1: Installing and Importing Libraries

Before running the code, ensure you have Python 3.8 or above installed. The required libraries can be installed using pip:

```
# Task 1: Installing and Importing Libraries
```

```
# It is assumed that the following libraries are already installed.
```

```
# If not, you would typically run:
```

```
# pip install requests skyfield numpy matplotlib ipywidgets
```

```
# Import necessary libraries
```

```
import requests
```

```
from skyfield.api import load, EarthSatellite, Topos
```

```
from skyfield.timelib import Time
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from datetime import datetime, timedelta
```

```
import pytz # For timezone handling
```

```
import pandas as pd # For Task 5: Tabulating results
```

```
# For interactive widgets (Task 6), these imports would be used in a  
# Jupyter environment:
```

```
# from ipywidgets import interact, Dropdown, FloatSlider, Button, Output
```

```
# from IPython.display import display, clear_output
```

```
print("All required libraries imported successfully.")

# Initialize a timescale for Skyfield

ts = load.timescale()

# Global variable to store ISS satellite object
iss_satellite = None
```

## **Task 2: Fetching ISS Two-Line Element (TLE) Data**

This code defines a function to fetch the current TLE data for the ISS from CelesTrak. TLEs are crucial for defining a satellite's orbit.

### **# Task 2: Fetching ISS Two-Line Element (TLE) Data**

```
def fetch_iss_tle():
    """
    Fetches the current Two-Line Element (TLE) data for the ISS from
    CelesTrak.

    Returns the TLE lines as a list of strings if successful, None otherwise.
    """

    # CelesTrak URL for ISS TLE data
    tle_url = "https://celestrak.org/NORAD/elements/stations.txt"

    try:
        response = requests.get(tle_url)

        response.raise_for_status() # Raise an HTTPError for bad responses
        (4xx or 5xx)

        tle_data = response.text.splitlines()

        # Find ISS TLE (International Space Station)
        iss_tle_lines = []
```



```

for i, line in enumerate(tle_data):

    if "ISS (ZARYA)" in line: # Or "INTERNATIONAL SPACE
        STATION"

        # TLE consists of three lines: Name, Line 1, Line 2

        if i + 2 < len(tle_data):

            iss_tle_lines = [tle_data[i], tle_data[i+1], tle_data[i+2]]

            break

if iss_tle_lines:

    print("Successfully fetched ISS TLE data.")

    # Data Validation (simple check for now)

    print(f"TLE Name: {iss_tle_lines[0]}")
    print(f"TLE Line 1: {iss_tle_lines[1]}")
    print(f"TLE Line 2: {iss_tle_lines[2]}")

    return iss_tle_lines

else:

    print("ISS TLE data not found in the fetched content.")

    return None

except requests.exceptions.RequestException as e:

    print(f"Error fetching TLE data: {e}")

    return None

except Exception as e:

    print(f"An unexpected error occurred: {e}")

    return None

```

**# Fetch ISS TLE data**

```

iss_tle = fetch_iss_tle()

# Load the ISS satellite object if TLE data is available

iss_satellite = None

if iss_tle:

    try:

        iss_satellite = EarthSatellite(iss_tle[1], iss_tle[2], iss_tle[0], ts)

        print("ISS satellite object created successfully.")

    except Exception as e:

        print(f"Error creating EarthSatellite object: {e}")

else:

    print("Could not retrieve ISS TLE data. Cannot proceed with satellite
    calculations.")

```

### **Task 3: Calculating ISS Pass Times for a Given Location**

This section defines functions to get observer coordinates from user input and then calculate when the ISS will pass over that location, including rise, culmination, and set times.

#### **# Task 3: Calculating ISS Pass Times for a Given Location**

```

def get_observer_location():

    """

    Prompts the user for latitude and longitude and validates the input.

    Returns a Topos object if input is valid, None otherwise.

    """

    while True:

```

```

try:

    latitude_str = input("Enter observer latitude (e.g., 28.6139 for
Delhi): ")

    longitude_str = input("Enter observer longitude (e.g., 77.2090 for
Delhi): ")

    latitude = float(latitude_str)

    longitude = float(longitude_str)

    if not (-90 <= latitude <= 90):

        print("Invalid latitude. Must be between -90 and 90.")

        continue

    if not (-180 <= longitude <= 180):

        print("Invalid longitude. Must be between -180 and 180.")

        continue

    return Topos(latitude_degrees=latitude,
longitude_degrees=longitude)

except ValueError:

    print("Invalid input. Please enter numerical values for latitude and
longitude.")

except Exception as e:

    print(f"An error occurred: {e}")

def calculate_iss_passes(observer_location, satellite, ts, days_ahead=2):

    """

    Calculates and displays ISS pass times for a given observer location.

    """

```

```

if not observer_location or not satellite:

    print("Observer location or satellite data is missing. Cannot calculate
    passes.")

    return

    print(f"\nCalculating ISS passes for location: Latitude
    {observer_location.latitude.degrees:.2f}°, Longitude
    {observer_location.longitude.degrees:.2f}°")

    t0 = ts.now()

    t1 = ts.utc(t0.utc_datetime() + timedelta(days=days_ahead))

    # Find events (rise, culmination, set)

    t, events = satellite.find_events(observer_location, t0, t1,
    altitude_degrees=10.0)

    print("\n--- ISS Pass Predictions ---")

    if len(events) == 0:

        print(f"No ISS passes found for the next {days_ahead} days above 10
        degrees altitude.")

        return

    # Process events

    for ti, event in zip(t, events):

        name = ('rise', 'culmination', 'set')[event]

        # Convert Skyfield time to a human-readable datetime object in local
        # timezone

        local_time = ti.astimezone(pytz.timezone('Asia/Kolkata')) # Example:
        India Standard Time (IST)

        print(f'{name.capitalize()}: {local_time.strftime('%Y-%m-%d %H:
        %M:%S %Z%Z')}")

```

```

# For more detailed pass information (like duration, peak altitude),
# you would typically group rise, culmination, and set events.
# Skyfield's find_events gives them sequentially.

# A more advanced approach would be to iterate through the events and
group them.

# For simplicity, we'll just list them as they appear.

# Example of how to get altitude at culmination (requires finding the
culmination event)

# This is a simplified approach, a full implementation would group rise/
culmination/set

# for each pass.

# For a given pass, you'd find the culmination event and then calculate
altitude.

# This example just shows how to get altitude for *any* time.
if events.size > 0:

    print("\n--- Example: Altitude at First Culmination ---")

    culmination_indices = np.where(events == 1)[0] # 1 is the index for
culmination event

    if culmination_indices.size > 0:

        first_culmination_time = t[culmination_indices[0]]

        difference = satellite - observer_location

        topocentric =
observer_location.at(first_culmination_time).observe(satellite)

        alt, az, distance = topocentric.altaz()

```

```

        print(f'Altitude at first culmination: {alt.degrees:.2f}°')
    else:
        print("No culmination events found in the current prediction
window.")

# Get observer location from user input
observer_loc = get_observer_location()

# Calculate ISS passes
calculate_iss_passes(observer_loc, iss_satellite, ts)

```

#### **Task 4: Visualizing the ISS Ground Track**

This code generates a plot of the ISS's path over the Earth for a specified time window using matplotlib.

##### **# Task 4: Visualizing the ISS Ground Track**

```

def plot_iss_ground_track(satellite, ts, minutes_ahead=120,
observer_location=None):
    """
    Plots the trajectory of the ISS over Earth for a specified time window.
    Optionally marks the observer's location.
    """
    if not satellite:
        print("Satellite data is missing. Cannot plot ground track.")
        return

```

```
print(f"\nGenerating ISS ground track for the next {minutes_ahead}
minutes...")
```

```
# Define a time range
```

```
t0 = ts.now()
```

```
t1 = ts.utc(t0.utc_datetime() + timedelta(minutes=minutes_ahead))
```

```
times = ts.linspace(t0, t1, minutes_ahead * 2) # Sample at 30-second
intervals
```

```
# Compute ISS sub-point (latitude, longitude) for each timestamp
```

```
geocentric = satellite.at(times)
```

```
latitudes, longitudes = geocentric.latitude.degrees,
geocentric.longitude.degrees
```

```
# Create the plot
```

```
plt.figure(figsize=(12, 8))
```

```
ax = plt.axes()
```

```
# Plot the world map
```

```
# Using a simple scatter plot for the world map for simplicity without
basemap/cartopy
```

```
# For a more detailed map, Basemap or Cartopy would be used, but they
require
```

```
# additional installations and setup which might be complex in a generic
environment.
```

```
# A basic world map can be approximated or assumed for visualization.
```

```
# For this example, we'll just plot the track on a blank canvas with grid
lines.
```

```
# If a user has Basemap/Cartopy installed, they can easily integrate it
here.
```

```
# Plot ISS trajectory
```

```
plt.plot(longitudes, latitudes, color='red', linewidth=2, label='ISS  
Ground Track')
```

```
plt.scatter(longitudes[-1], latitudes[-1], color='blue', s=100, marker='o',  
label='Current ISS Position', zorder=5)
```

```
# Add observer location if provided
```

```
if observer_location:
```

```
    plt.scatter(observer_location.longitude.degrees,  
observer_location.latitude.degrees,  
  
                color='green', s=150, marker='X', label='Observer Location',  
zorder=5)
```

```
# Set plot limits and labels
```

```
plt.xlim(-180, 180)
```

```
plt.ylim(-90, 90)
```

```
plt.xlabel("Longitude (°)")
```

```
plt.ylabel("Latitude (°)")
```

```
plt.title(f"ISS Ground Track (Next {minutes_ahead} Minutes) from  
{t0.utc_datetime().strftime('%Y-%m-%d %H:%M:%S UTC')}")
```

```
plt.grid(True, linestyle='--', alpha=0.7)
```

```
plt.legend()
```

```
plt.axvline(0, color='gray', linestyle=':', linewidth=0.5)
```

```
plt.axhline(0, color='gray', linestyle=':', linewidth=0.5)
```

```
# Add a simple representation of the Earth's outline
```

```
plt.gca().set_aspect('equal', adjustable='box')
```

```
plt.show()
```



```
print("ISS ground track plot generated.")
```

```
# Plot the ISS ground track
```

```
plot_iss_ground_track(iss_satellite, ts, observer_location=observer_loc)
```

## **Task 5: Predicting Passes Over Multiple Locations**

This section automates pass predictions for a list of predefined cities and tabulates the results using pandas.

```
# Task 5: Predicting Passes Over Multiple Locations
```

```
import pandas as pd # For tabulating results
```

```
def predict_passes_for_multiple_locations(satellite, ts, locations_data,
                                          days_ahead=1):
```

```
    """
```

```
    Automates pass predictions for a list of cities or coordinates.
```

```
    locations_data: A list of dictionaries, e.g.,
```

```
        [{ 'name': 'Delhi', 'latitude': 28.6139, 'longitude': 77.2090}, ...]
```

```
    """
```

```
    if not satellite:
```

```
        print("Satellite data is missing. Cannot predict passes for multiple
              locations.")
```

```
    return
```

```
    print(f"\nPredicting ISS passes for multiple locations for the next
          {days_ahead} day(s)...\n")
```

```
    results = []
```

```
    t0 = ts.now()
```

```

t1 = ts.utc(t0.utc_datetime() + timedelta(days=days_ahead))

for loc in locations_data:

    name = loc['name']

    latitude = loc['latitude']

    longitude = loc['longitude']

    observer = Topos(latitude_degrees=latitude,
                      longitude_degrees=longitude)

    print(f"\n--- Processing {name} ({latitude:.2f}°, {longitude:.2f}°)
    ---")

    t, events = satellite.find_events(observer, t0, t1,
                                      altitude_degrees=10.0)

    if len(events) == 0:

        results.append({

            'Location': name,

            'Rise Time (IST)': 'No pass',

            'Culmination (IST)': 'No pass',

            'Set Time (IST)': 'No pass',

            'Duration (min)': 'N/A',

            'Peak Altitude (deg)': 'N/A'

        })

        print(f"No ISS passes found for {name}.")

        continue

# Group events into passes (rise, culmination, set)

passes = []

```

```

current_pass = {}

for ti, event in zip(t, events):

    event_name = ('rise', 'culmination', 'set')[event]

    local_time_ist = ti.astimezone(pytz.timezone('Asia/Kolkata'))

    if event_name == 'rise':

        current_pass = {'Location': name, 'Rise Time (IST)':
local_time_ist}

        elif event_name == 'culmination' and 'Rise Time (IST)' in
current_pass:

            current_pass['Culmination (IST)'] = local_time_ist

            # Calculate peak altitude at culmination

            difference = satellite - observer

            topocentric = observer.at(ti).observe(satellite)

            alt, az, distance = topocentric.altaz()

            current_pass['Peak Altitude (deg)'] = f"{alt.degrees:.2f}"

            elif event_name == 'set' and 'Rise Time (IST)' in current_pass and
'Culmination (IST)' in current_pass:

                current_pass['Set Time (IST)'] = local_time_ist

                duration = (current_pass['Set Time (IST)'] - current_pass['Rise
Time (IST)']).total_seconds() / 60

                current_pass['Duration (min)'] = f"{duration:.2f}"

                passes.append(current_pass)

                current_pass = {} # Reset for next pass

# Add collected passes to overall results

```

for p in passes:

*# Format datetime objects for display*

p['Rise Time (IST)'] = p['Rise Time (IST)'].strftime('%Y-%m-%d %H:%M:%S %Z%z')

p['Culmination (IST)'] = p['Culmination (IST)'].strftime('%Y-%m-%d %H:%M:%S %Z%z')

p['Set Time (IST)'] = p['Set Time (IST)'].strftime('%Y-%m-%d %H:%M:%S %Z%z')

results.append(p)

*# Tabulate results using pandas DataFrame*

if results:

df = pd.DataFrame(results)

print("\n--- ISS Pass Predictions for Multiple Locations ---")

print(df.to\_string(index=False))

*# Optionally save to CSV*

*# df.to\_csv("iss\_passes\_multi\_location.csv", index=False)*

*# print("\nResults saved to iss\_passes\_multi\_location.csv")*

else:

print("\nNo pass predictions generated for any location.")

*# Example locations data (you can load this from a CSV/JSON file)*

example\_locations = [

{'name': 'Delhi, India', 'latitude': 28.6139, 'longitude': 77.2090},

{'name': 'Bengaluru, India', 'latitude': 12.9716, 'longitude': 77.5946},

{'name': 'New York, USA', 'latitude': 40.7128, 'longitude': -74.0060},

```
{'name': 'London, UK', 'latitude': 51.5074, 'longitude': -0.1278},  
{'name': 'Sydney, Australia', 'latitude': -33.8688, 'longitude': 151.2093}  
]  
  
# Predict passes for multiple locations  
  
predict_passes_for_multiple_locations(iss_satellite, ts, example_locations)
```

### **Task 6: Interactive Exploration (Jupyter Environment Specific)**

This task enhances the tracking capabilities with interactive widgets, primarily designed for a Jupyter Notebook environment.

The code below provides the structure. To run this, you would typically uncomment the ipywidgetsimports and the display/on\_click lines in a live Jupyter session.

#### **# Task 6: Interactive ISS Exploration for Jupyter Notebook Environment**

##### **# Import required libraries**

```
from ipywidgets import interact, Dropdown, FloatSlider, Button, Output,  
VBox, HBox
```

```
from IPython.display import display, clear_output
```

```
import matplotlib.pyplot as plt
```

```
from skyfield.api import load, EarthSatellite, Topos
```

```
from datetime import datetime, timedelta
```

```
import pytz
```

```
import numpy as np
```

```
import pandas as pd
```

```
import requests
```

```
# Initialize Skyfield timescale
```

```
ts = load.timescale()
```

```
def fetch_iss_tle():
```

```
    """
```

```
    Fetch current ISS Two-Line Element (TLE) data from Celestrak.
```

```
    Returns:
```

```
        list: [name, line1, line2] if successful, None if failed
```

```
    """
```

```
    try:
```

```
        url = "https://celestrak.org/NORAD/elements/stations.txt"
```

```
        response = requests.get(url, timeout=10)
```

```
        response.raise_for_status()
```

```
        lines = response.text.strip().split('\n')
```

```
        # Find ISS data (usually named "ISS (ZARYA)")
```

```
        for i in range(0, len(lines) - 2, 3):
```

```
            if "ISS" in lines[i].upper():
```

```
                return [lines[i].strip(), lines[i+1].strip(), lines[i+2].strip()]
```

```
        print("ISS TLE data not found in response")
```

```
        return None
```

```
    except requests.RequestException as e:
```

```
        print(f"Error fetching ISS TLE data: {e}")
```

```
        return None
```

```
    except Exception as e:
```

```
print(f"Unexpected error: {e}")
```

```
return None
```

```
def plot_iss_ground_track(satellite, timescale, minutes_ahead=120,  
observer_location=None):
```

```
    """
```

Plot ISS ground track on world map.

Parameters:

satellite: EarthSatellite object

timescale: Skyfield timescale object

minutes\_ahead (int): Minutes to plot ahead from current time

observer\_location: Topos object for observer location (optional)

```
    """
```

```
# Generate time points for the ground track
```

```
t0 = timescale.now()
```

```
t1 = timescale.utc(t0.utc_datetime() +  
timedelta(minutes=minutes_ahead))
```

```
# Create time array with 1-minute intervals
```

```
times = timescale.utc(  
    t0.utc_datetime() + timedelta(minutes=i)  
    for i in range(0, minutes_ahead + 1, 1)  
)
```

```
# Calculate ISS positions
```

```
geocentric = satellite.at(times)
```

```
subpoint = geocentric.subpoint()
```

```

latitudes = subpoint.latitude.degrees
longitudes = subpoint.longitude.degrees

# Create the plot
plt.figure(figsize=(12, 6))

# Plot world map outline (simplified)
plt.axhline(y=0, color='lightgray', linestyle='-', alpha=0.5) # Equator
plt.axvline(x=0, color='lightgray', linestyle='-', alpha=0.5) # Prime
meridian

# Plot ISS ground track
plt.plot(longitudes, latitudes, 'b-', linewidth=2, label='ISS Ground
Track')

plt.scatter(longitudes[0], latitudes[0], color='green', s=100,
            marker='o', label='Current Position', zorder=5)

plt.scatter(longitudes[-1], latitudes[-1], color='red', s=100,
            marker='s', label=f'Position in {minutes_ahead} min', zorder=5)

# Plot observer location if provided
if observer_location:
    obs_lat = observer_location.latitude.degrees
    obs_lon = observer_location.longitude.degrees
    plt.scatter(obs_lon, obs_lat, color='orange', s=150,
                marker='*', label='Observer Location', zorder=5)

# Set plot properties
plt.xlim(-180, 180)
plt.ylim(-90, 90)
plt.xlabel('Longitude (degrees)')

```



```

plt.ylabel('Latitude (degrees)')
plt.title(f'ISS Ground Track - Next {minutes_ahead} Minutes')
plt.grid(True, alpha=0.3)
plt.legend()
# Add grid lines for continents reference
for lat in [-60, -30, 30, 60]:
    plt.axhline(y=lat, color='lightgray', linestyle='--', alpha=0.3)
for lon in [-120, -60, 60, 120]:
    plt.axvline(x=lon, color='lightgray', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()

```

### # Initialize ISS satellite object

```

print("Fetching current ISS orbital data...")
iss_tle_data = fetch_iss_tle()
iss_satellite = None
if iss_tle_data:
    iss_satellite = EarthSatellite(iss_tle_data[1], iss_tle_data[2],
iss_tle_data[0], ts)
    print(f'Successfully loaded ISS data: {iss_tle_data[0]}')
else:
    print(" Failed to fetch ISS TLE data. Interactive features will be
limited.")

```

### # Define example observation locations

```

example_locations = [
    {'name': 'Delhi, India', 'latitude': 28.6139, 'longitude': 77.2090},

```

```

{'name': 'Bengaluru, India', 'latitude': 12.9716, 'longitude': 77.5946},
{'name': 'Mumbai, India', 'latitude': 19.0760, 'longitude': 72.8777},
{'name': 'New York, USA', 'latitude': 40.7128, 'longitude': -74.0060},
{'name': 'London, UK', 'latitude': 51.5074, 'longitude': -0.1278},
{'name': 'Tokyo, Japan', 'latitude': 35.6762, 'longitude': 139.6503},
{'name': 'Sydney, Australia', 'latitude': -33.8688, 'longitude': 151.2093},
{'name': 'São Paulo, Brazil', 'latitude': -23.5505, 'longitude': -46.6333}
]

```

**# Create location dictionary for easy lookup**

```

locations_dict = {

    loc['name']: Topos(latitude_degrees=loc['latitude'],
longitude_degrees=loc['longitude'])

    for loc in example_locations

}

location_names = [loc['name'] for loc in example_locations]

```

**# Create output widget for displaying results**

```

output_area = Output()

def interactive_iss_tracker(selected_location_name, minutes_ahead_plot,
days_ahead_passes):

    """

```

Main interactive function called by widgets.

Parameters:

selected\_location\_name (str): Name of selected observation location

minutes\_ahead\_plot (float): Minutes ahead for ground track plot

days\_ahead\_passes (float): Days ahead for pass predictions

```
"""
```

```
with output_area:
```

```
    clear_output(wait=True)
```

```
    if not iss_satellite:
```

```
        print("ISS satellite data not available.")
```

```
        print("Please check internet connection and try refreshing the TLE  
data.")
```

```
    return
```

```
    selected_topos = locations_dict[selected_location_name]
```

```
    selected_location = next(loc for loc in example_locations if  
loc['name'] == selected_location_name)
```

```
    print(f"ISS Tracking Information for {selected_location_name}")
```

```
    print(f"Location: {selected_location['latitude']:.4f}°N,  
{selected_location['longitude']:.4f}°E")
```

```
    print("=" * 60)
```

```
# Calculate current ISS position
```

```
    current_time = ts.now()
```

```
    geocentric = iss_satellite.at(current_time)
```

```
    subpoint = geocentric.subpoint()
```

```
    print(f"\n Current ISS Position:")
```

```
    print(f"  Latitude: {subpoint.latitude.degrees:.4f}°")
```

```
    print(f"  Longitude: {subpoint.longitude.degrees:.4f}°")
```

```
    print(f"  Altitude: {subpoint.elevation.km:.1f} km")
```

```
    print(f"  Time (UTC): {current_time.utc_strftime('%Y-%m-%d %H:  
%M:%S')}")
```

## # Calculate ISS pass predictions

```
print(f"\n🔭 ISS Pass Predictions (Next {days_ahead_passes}
days)")

print("-" * 50)

t0 = ts.now()

t1 = ts.utc(t0.utc_datetime() + timedelta(days=days_ahead_passes))

try:
    t, events = iss_satellite.find_events(selected_topos, t0, t1,
altitude_degrees=10.0)

    if len(events) == 0:

        print(f"No ISS passes found above 10° elevation for
{selected_location_name}")

        print(f"in the next {days_ahead_passes} days.")
    else:

        passes_data = []
        current_pass = {}

        for ti, event in zip(t, events):

            event_name = ('Rise', 'Culmination', 'Set')[event]

            # Convert to local timezone (IST for Indian locations, UTC
otherwise)

            if 'India' in selected_location_name:

                local_time = ti.astimezone(pytz.timezone('Asia/Kolkata'))

                tz_name = 'IST'
            else:

                local_time = ti.utc_datetime()

                tz_name = 'UTC'
```

```

time_str = local_time.strftime('%Y-%m-%d %H:%M:%S') + f
{tz_name}'

if event_name == 'Rise':

    current_pass = {

        'Date': local_time.strftime('%Y-%m-%d'),

        'Rise Time': local_time.strftime('%H:%M:%S')

    }

elif event_name == 'Culmination' and current_pass:

    # Calculate peak altitude and azimuth

    difference = iss_satellite - selected_topos

    topocentric = difference.at(ti)

    alt, az, distance = topocentric.altaz()

    current_pass['Max Alt'] = f'{alt.degrees:.1f}°'

    current_pass['Azimuth'] = f'{az.degrees:.0f}°'

    current_pass['Culmination'] = local_time.strftime('%H:%M:
%S')

elif event_name == 'Set' and current_pass:

    current_pass['Set Time'] = local_time.strftime('%H:%M:
%S')

    # Calculate duration

    if 'Rise Time' in current_pass and 'Set Time' in
current_pass:

        rise_dt = datetime.strptime(current_pass['Rise Time'],
'%H:%M:%S')

        set_dt = datetime.strptime(current_pass['Set Time'], '%H:
%M:%S')

```

```

        if set_dt < rise_dt: # Handle day rollover
            set_dt += timedelta(days=1)
            duration = set_dt - rise_dt
            current_pass['Duration'] = str(duration).split('.')[0] #
Remove microseconds
            passes_data.append(current_pass)
            current_pass = {}

    if passes_data:
        df_passes = pd.DataFrame(passes_data)
        print(df_passes.to_string(index=False))
        # Highlight best passes (high altitude)
        if 'Max Alt' in df_passes.columns:
            best_passes = df_passes[df_passes['Max Alt'].str.replace('°',
").astype(float) > 45]
            if not best_passes.empty:
                print(f"\n Excellent passes (>45° elevation):
{len(best_passes)}")
            else:
                print("No complete pass sequences found.")
except Exception as e:
    print(f"Error calculating passes: {e}")
    # Display ground track plot
    print(f"\n Ground Track Visualization")
    print("-" * 30)
    plot_iss_ground_track(iss_satellite, ts,

```

```
minutes_ahead=int(minutes_ahead_plot),  
observer_location=selected_topos)
```

```
# Create interactive widgets
```

```
print("\n Interactive ISS Tracker Controls")
```

```
print("=" * 40)
```

```
location_dropdown = Dropdown(  
    options=location_names,  
    value=location_names[0],  
    description='Location:',  
    style={'description_width': 'initial'}  
)
```

```
plot_minutes_slider = FloatSlider(  
    min=30, max=240, step=10, value=120,  
    description='Plot Minutes:',  
    style={'description_width': 'initial'},  
    readout_format='.0f'  
)
```

```
pass_days_slider = FloatSlider(  
    min=0.5, max=7, step=0.5, value=2,  
    description='Pass Days:',  
    style={'description_width': 'initial'},  
    readout_format='.1f'  
)
```

```
update_button = Button(
    description="Update Data",
    button_style='primary',
    tooltip='Click to refresh ISS data and predictions'
)

refresh_tle_button = Button(
    description="Refresh TLE",
    button_style='info',
    tooltip='Fetch latest ISS orbital elements'
)
```

### # Widget event handlers

```
def on_update_click(button):
    """Handle update button click."""
    interactive_iss_tracker(
        location_dropdown.value,
        plot_minutes_slider.value,
        pass_days_slider.value
    )

def on_refresh_tle_click(button):
    """Handle TLE refresh button click."""
    global iss_satellite
    with output_area:
        clear_output(wait=True)
        print("Refreshing ISS orbital data...")
```



```

iss_tle_data = fetch_iss_tle()

if iss_tle_data:

    iss_satellite = EarthSatellite(iss_tle_data[1], iss_tle_data[2],
iss_tle_data[0], ts)

    with output_area:

        clear_output(wait=True)

        print(f' Successfully updated ISS data: {iss_tle_data[0]}')

        print("Click 'Update Data' to refresh predictions.")

else:

    with output_area:

        clear_output(wait=True)

        print(" Failed to refresh TLE data. Using cached data if
available.")

# Bind event handlers

update_button.on_click(on_update_click)
refresh_tle_button.on_click(on_refresh_tle_click)

# Create widget layout

controls = HBox([location_dropdown, plot_minutes_slider,
pass_days_slider])

buttons = HBox([update_button, refresh_tle_button])

widget_box = VBox([controls, buttons, output_area])

# Display the interface

display(widget_box)

```

```
# Initial data load
```

```
if iss_satellite:
```

```
    print("ISS Interactive Tracker Ready!")
```

```
    print("Select your location and click 'Update Data' to begin.")
```

```
    print("\nNote: TLE data is automatically fetched. Use 'Refresh TLE' for  
latest orbital elements.")
```

```
else:
```

```
    print("Please click 'Refresh TLE' to fetch ISS orbital data before using  
the tracker.")
```

### **Task 7: Advanced Analysis – Orbital Drift over Time**

This task focuses on analyzing long-term changes in ISS orbital elements. It conceptually requires historical TLE data.

For demonstration, the code simulates data, but in a real scenario, you would download and process historical TLE files (e.g., from Space-Track.org).

```
# Task 7: Advanced Analysis – Orbital Drift over Time
```

```
# This task requires downloading historical TLE data, which is a manual  
step
```

```
# or involves programmatic fetching from an archive (e.g., Space-  
Track.org, which requires registration).
```

```
# For demonstration purposes, we will simulate or assume a small set of  
historical TLEs
```

```
# or provide the structure for how to process them if they were available.
```

```
# If you have historical TLE files (e.g., 'iss_2023-01-01.txt',  
'iss_2023-01-02.txt'),
```

```

# you would iterate through them.

# Example structure for processing historical TLEs:

# import os

# from skyfield.api import load, EarthSatellite

# import matplotlib.pyplot as plt

# from datetime import datetime, timedelta

# ts = load.timescale() # Ensure timescale is loaded

def analyze_orbital_drift(tle_files_directory="historical_tles"):
    """
    Analyzes long-term changes in ISS orbital elements from historical TLE
    data.

    tle_files_directory: Path to a directory containing historical TLE files.
    """

    print("\n--- Advanced Analysis: Orbital Drift over Time ---")

    print("This function assumes historical TLE files are available in the
    specified directory.")

    print("Please ensure you have downloaded and placed TLE files (e.g.,
    from Space-Track.org archive).")

    # For demonstration, we'll use a placeholder for historical data.

    # In a real scenario, you would read TLEs from files.

    # Let's simulate some data points for demonstration

    dates = []

    inclinations = []

    eccentricities = []

```

```

periods = [] # Orbital period in minutes

# Simulate data for a month (30 days)
start_date = datetime(2023, 1, 1)

for i in range(30):
    current_date = start_date + timedelta(days=i)
    dates.append(current_date)

    # Simulate slight variations for demonstration
    inclinations.append(51.64 + np.random.uniform(-0.01, 0.01))
    eccentricities.append(0.0002 + np.random.uniform(-0.00005,
0.00005))

    # Orbital period for ISS is roughly 92 minutes
    periods.append(92.0 + np.random.uniform(-0.1, 0.1))

# In a real scenario, you would parse TLE files like this:
for filename in sorted(os.listdir(tle_files_directory)):
    if filename.endswith(".txt"):
        file_path = os.path.join(tle_files_directory, filename)
        with open(file_path, 'r') as f:
            tle_lines = f.readlines()

            # Assuming the file contains one ISS TLE set
            # You'd need to parse the specific lines for the ISS
            if len(tle_lines) >= 3:
                try:
                    satellite = EarthSatellite(tle_lines[1], tle_lines[2],
tle_lines[0].strip(), ts)

```

```

# Extract date from filename or TLE

date_str = filename.replace('iss_', '').replace('.txt', '')
dates.append(datetime.strptime(date_str, '%Y-%m-%d'))

inclinations.append(satellite.inclination.degrees)

eccentricities.append(satellite.eccentricity)

# Calculate orbital period (approximate for demonstration)

# Period = 2 * pi * sqrt(a^3 / mu), where a is semi-major
axis, mu is standard gravitational parameter

# Skyfield's satellite object has a period attribute or can be
derived.

# For simplicity, we'll use a fixed value or derive from
mean motion if available in TLE.

# A common way to get period from TLE is from the 'mean
motion' (revolutions per day).

# period_minutes = 1440 / satellite.model.no * 1 # no is
mean motion in revs/day

periods.append(92.0) # Placeholder

except Exception as e:

    print(f"Error processing {filename}: {e}")

if not dates:

    print("No historical TLE data processed. Cannot plot trends.")

    return

# Plotting trends

plt.figure(figsize=(14, 10))

```

```
plt.subplot(3, 1, 1)

plt.plot(dates, inclinations, marker='o', linestyle='-', markersize=4)

plt.title('ISS Orbital Inclination Over Time')

plt.ylabel('Inclination (degrees)')

plt.grid(True)

plt.xticks(rotation=45)

plt.subplot(3, 1, 2)

plt.plot(dates, eccentricities, marker='o', linestyle='-', markersize=4,
color='orange')

plt.title('ISS Orbital Eccentricity Over Time')

plt.ylabel('Eccentricity')

plt.grid(True)

plt.xticks(rotation=45)

plt.subplot(3, 1, 3)

plt.plot(dates, periods, marker='o', linestyle='-', markersize=4,
color='green')

plt.title('ISS Orbital Period Over Time')

plt.ylabel('Period (minutes)')

plt.xlabel('Date')

plt.grid(True)

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()
```

```
print("\n--- Interpretation Questions ---")
```

```
print("1. What factors (e.g., atmospheric drag, reboost maneuvers)  
explain observed trends?")
```

```
print(" - Atmospheric drag constantly pulls the ISS lower, causing its  
orbital period to decrease slightly and its altitude to decay. This would  
manifest as a slight decrease in orbital period over time if not for  
reboosts.")
```

```
print(" - Reboost maneuvers (using thrusters from visiting spacecraft or  
the ISS itself) periodically raise the ISS's altitude and increase its orbital  
period, counteracting drag. These would appear as sharp increases in the  
plots.")
```

```
print(" - Solar activity can influence atmospheric density, affecting  
drag and thus the rate of orbital decay.")
```

```
print(" - Inclination and eccentricity are generally more stable but can  
be subtly affected by various gravitational perturbations and reboosts.")
```

```
print("2. How might this affect ground-track predictions?")
```

```
print(" - Changes in orbital period directly affect the timing of passes  
over specific locations. A shorter period means the ISS completes an orbit  
faster, causing it to appear earlier than predicted if using outdated TLEs.")
```

```
print(" - Variations in inclination can slightly shift the latitudinal extent  
of the ground track, meaning it might pass over slightly different regions.")
```

```
print(" - Changes in eccentricity (though typically very small for ISS)  
would affect the shape of the orbit, leading to variations in altitude and  
speed at different points, which could subtly alter pass times and  
visibility.")
```

```
print(" - Accurate and up-to-date TLEs are crucial for precise ground-  
track predictions because these orbital elements are constantly changing  
due to various forces.")
```

```
# Call the function to analyze orbital drift (using simulated data for now)
```

```
analyze_orbital_drift()
```

## **Task 8: Building a Real-Time Dashboard Application (Conceptual)**

This section outlines the conceptual structure for building a real-time web application for ISS tracking. It describes the backend API and frontend visualization components but does not provide a runnable web application, as that would require a full Flask/Streamlit setup and deployment environment.

```
# Real-Time ISS Dashboard Application
```

```
# Complete implementation with Flask backend and HTML frontend
```

```
from flask import Flask, jsonify, render_template_string
```

```
from flask_cors import CORS
```

```
from skyfield.api import load, EarthSatellite, Topos
```

```
from datetime import datetime, timedelta
```

```
import requests
```

```
import json
```

```
import threading
```

```
import time
```

```
import pytz
```

```
# Initialize Flask app
```

```
app = Flask(__name__)
```

```
CORS(app) # Enable CORS for frontend requests
```

```
# Global variables
```



```

ts = load.timescale()

iss_satellite = None

tle_last_updated = None

update_lock = threading.Lock()

# Major cities for pass predictions

MAJOR_CITIES = [

    {'name': 'New York', 'lat': 40.7128, 'lon': -74.0060, 'tz': 'America/
New_York'},

    {'name': 'London', 'lat': 51.5074, 'lon': -0.1278, 'tz': 'Europe/London'},

    {'name': 'Tokyo', 'lat': 35.6762, 'lon': 139.6503, 'tz': 'Asia/Tokyo'},

    {'name': 'Sydney', 'lat': -33.8688, 'lon': 151.2093, 'tz': 'Australia/
Sydney'},

    {'name': 'Delhi', 'lat': 28.6139, 'lon': 77.2090, 'tz': 'Asia/Kolkata'},

    {'name': 'São Paulo', 'lat': -23.5505, 'lon': -46.6333, 'tz': 'America/
Sao_Paulo'},

    {'name': 'Cairo', 'lat': 30.0444, 'lon': 31.2357, 'tz': 'Africa/Cairo'},

    {'name': 'Los Angeles', 'lat': 34.0522, 'lon': -118.2437, 'tz': 'America/
Los_Angeles'}

]

def fetch_iss_tle():

    """

    Fetch current ISS Two-Line Element data from NORAD.

    Returns:

    EarthSatellite: Skyfield satellite object or None if failed

```

"""

try:

```
print("Fetching latest ISS TLE data...")
```

```
url = "https://celestrak.org/NORAD/elements/stations.txt"
```

```
response = requests.get(url, timeout=15)
```

```
response.raise_for_status()
```

```
lines = response.text.strip().split('\n')
```

```
# Find ISS data
```

```
for i in range(0, len(lines) - 2, 3):
```

```
    if "ISS" in lines[i].upper() and "ZARYA" in lines[i].upper():
```

```
        satellite = EarthSatellite(
```

```
            lines[i+1].strip(),
```

```
            lines[i+2].strip(),
```

```
            lines[i].strip(),
```

```
            ts
```

```
        )
```

```
        print(f" Successfully loaded: {lines[i].strip()}")
```

```
        return satellite
```

```
print("ISS TLE data not found in response")
```

```
return None
```

except Exception as e:

```
print(f"Error fetching ISS TLE: {e}")
```

```
return None
```

```

def update_iss_data():
    """Background task to update ISS TLE data periodically."""
    global iss_satellite, tle_last_updated
    while True:
        try:
            with update_lock:
                new_satellite = fetch_iss_tle()
                if new_satellite:
                    iss_satellite = new_satellite
                    tle_last_updated = datetime.utcnow()
                    print(f"TLE updated at {tle_last_updated.strftime('%Y-%m-%d %H:%M:%S')} UTC")
            except Exception as e:
                print(f"Error in update thread: {e}")
        # Update every 6 hours
        time.sleep(6 * 3600)

```

```

def calculate_next_passes(city_data, days_ahead=2):

```

```

    """

```

Calculate next ISS passes for a city.

Args:

city\_data (dict): City information with lat, lon, tz

days\_ahead (int): Number of days to look ahead

Returns:

list: List of pass dictionaries

"""

if not iss\_satellite:

return []

try:

location = Topos(latitude\_degrees=city\_data['lat'],

longitude\_degrees=city\_data['lon'])

t0 = ts.now()

t1 = ts.utc(t0.utc\_datetime() + timedelta(days=days\_ahead))

t, events = iss\_satellite.find\_events(location, t0, t1,  
altitude\_degrees=10.0)

passes = []

current\_pass = {}

for ti, event in zip(t, events):

event\_name = ('rise', 'culmination', 'set')[event]

# Convert to city's timezone

city\_tz = pytz.timezone(city\_data['tz'])

local\_time = ti.astimezone(city\_tz)

if event\_name == 'rise':

current\_pass = {

'date': local\_time.strftime('%Y-%m-%d'),

'rise\_time': local\_time.strftime('%H:%M:%S'),

```

        'rise_timestamp': ti.ut1
    }

elif event_name == 'culmination' and current_pass:
    # Calculate maximum altitude
    difference = iss_satellite - location
    topocentric = difference.at(ti)
    alt, az, distance = topocentric.altaz()
    current_pass.update({
        'max_altitude': round(alt.degrees, 1),
        'azimuth': round(az.degrees, 0),
        'culmination_time': local_time.strftime('%H:%M:%S')
    })

elif event_name == 'set' and current_pass:
    current_pass.update({
        'set_time': local_time.strftime('%H:%M:%S'),
        'timezone': city_data['tz'].split('/')[1]
    })
    # Calculate duration
    if 'rise_time' in current_pass:
        rise_dt = datetime.strptime(current_pass['rise_time'], '%H:
%M:%S')
        set_dt = datetime.strptime(current_pass['set_time'], '%H:%M:
%S')
        if set_dt < rise_dt: # Handle midnight crossing

```

```

        set_dt += timedelta(days=1)

        duration = set_dt - rise_dt

        current_pass['duration'] = str(duration).split('.')[0]

        passes.append(current_pass)

        current_pass = {}

    return sorted(passes, key=lambda x: x.get('rise_timestamp', 0))[:5] #
Next 5 passes

except Exception as e:

    print(f"Error calculating passes for {city_data['name']}: {e}")

    return []

```

## # API Routes

```

@app.route('/')
def index():

    """Serve the main dashboard page."""

    return render_template_string(HTML_TEMPLATE)

@app.route('/api/iss/position')
def get_iss_position():

    """Get current ISS position."""

    if not iss_satellite:

        return jsonify({

            'error': 'ISS orbital data not available',

            'status': 'error'

        }), 503

```

try:

```
# Get current ISS position
```

```
t = ts.now()
```

```
geocentric = iss_satellite.at(t)
```

```
subpoint = geocentric.subpoint()
```

```
# Calculate velocity (approximate)
```

```
t_prev = ts.utc(t.utc_datetime() - timedelta(minutes=1))
```

```
geocentric_prev = iss_satellite.at(t_prev)
```

```
subpoint_prev = geocentric_prev.subpoint()
```

```
# Calculate distance traveled in 1 minute
```

```
from math import radians, sin, cos, sqrt, atan2
```

```
lat1, lon1 = radians(subpoint_prev.latitude.degrees),  
radians(subpoint_prev.longitude.degrees)
```

```
lat2, lon2 = radians(subpoint.latitude.degrees),  
radians(subpoint.longitude.degrees)
```

```
dlat = lat2 - lat1
```

```
dlon = lon2 - lon1
```

```
a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2
```

```
c = 2 * atan2(sqrt(a), sqrt(1-a))
```

```
distance = 6371 * c # Earth radius in km
```

```
velocity = distance # km/min, convert to km/h
```

```
return jsonify({
```

```
    'latitude': round(subpoint.latitude.degrees, 6),
```

```
    'longitude': round(subpoint.longitude.degrees, 6),
```

```

        'altitude': round(subpoint.elevation.km, 2),
        'velocity': round(velocity * 60, 2), # km/h
        'timestamp': t.utcnow().isoformat() + 'Z',
        'tle_updated': tle_last_updated.isoformat() + 'Z' if tle_last_updated
else None,
        'status': 'success'
    })
except Exception as e:
    return jsonify({
        'error': f'Error calculating ISS position: {str(e)}',
        'status': 'error'
    }), 500

@app.route('/api/iss/ground-track')
def get_ground_track():
    """Get ISS ground track for next 90 minutes."""
    if not iss_satellite:
        return jsonify({'error': 'ISS data not available'}), 503
    try:
        t0 = ts.now()
        minutes_ahead = 90
        # Generate time points every 2 minutes
        times = []
        for i in range(0, minutes_ahead + 1, 2):

```



```

        times.append(ts.utc(t0.utc_datetime() + timedelta(minutes=i)))

positions = []

for t in times:

    geocentric = iss_satellite.at(t)
    subpoint = geocentric.subpoint()
    positions.append({
        'lat': round(subpoint.latitude.degrees, 4),
        'lon': round(subpoint.longitude.degrees, 4),
        'timestamp': t.utc_datetime().isoformat() + 'Z'
    })

return jsonify({
    'positions': positions,
    'status': 'success'
})

except Exception as e:

    return jsonify({'error': str(e)}), 500

@app.route('/api/iss/passes')
def get_iss_passes():
    """Get ISS pass predictions for major cities."""

    if not iss_satellite:

        return jsonify({'error': 'ISS data not available'}), 503

    try:

        all_passes = {}

```

```
for city in MAJOR_CITIES:
```

```
    passes = calculate_next_passes(city, days_ahead=3)
```

```
    all_passes[city['name']] = {
```

```
        'city_info': city,
```

```
        'passes': passes
```

```
    }
```

```
return jsonify({
```

```
    'passes': all_passes,
```

```
    'status': 'success',
```

```
    'generated_at': datetime.utcnow().isoformat() + 'Z'
```

```
})
```

```
except Exception as e:
```

```
    return jsonify({'error': str(e)}), 500
```

```
@app.route('/api/iss/status')
```

```
def get_system_status():
```

```
    """Get system status and diagnostics."""
```

```
    return jsonify({
```

```
        'iss_data_available': iss_satellite is not None,
```

```
        'tle_last_updated': tle_last_updated.isoformat() + 'Z' if  
tle_last_updated else None,
```

```
        'server_time': datetime.utcnow().isoformat() + 'Z',
```

```
        'cities_monitored': len(MAJOR_CITIES),
```

```
        'status': 'operational' if iss_satellite else 'degraded'
```

```
}}
```

## # HTML Template for the dashboard

```
HTML_TEMPLATE = "
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-  
scale=1.0">
```

```
  <title> Real-Time ISS Tracker</title>
```

```
  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/  
leaflet.css" />
```

```
  <style>
```

```
    * { margin: 0; padding: 0; box-sizing: border-box; }
```

```
    body {
```

```
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

```
      background: linear-gradient(135deg, #0f0f23 0%, #1a1a2e 100%);
```

```
      color: #ffffff;
```

```
      overflow-x: hidden;
```

```
    }
```

```
    .header {
```

```
      background: rgba(0,0,0,0.8);
```

```
      padding: 1rem;
```

```
      text-align: center;
```

```
border-bottom: 2px solid #00ff88;
}

.header h1 {
  font-size: 2.5rem;
  background: linear-gradient(45deg, #00ff88, #00ccff);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  margin-bottom: 0.5rem;
}

.status-bar {
  display: flex;
  justify-content: center;
  gap: 2rem;
  margin-top: 1rem;
  flex-wrap: wrap;
}

.status-item {
  background: rgba(255,255,255,0.1);
  padding: 0.5rem 1rem;
  border-radius: 20px;
  border: 1px solid rgba(0,255,136,0.3);
}

.container {
```

```
display: grid;

grid-template-columns: 1fr 400px;

height: calc(100vh - 140px);

gap: 1rem;

padding: 1rem;
}

.map-container {

position: relative;

border-radius: 10px;

overflow: hidden;

border: 2px solid #00ff88;

}

#map {

height: 100%;

width: 100%;

}

.sidebar {

background: rgba(0,0,0,0.8);

border-radius: 10px;

padding: 1.5rem;

border: 1px solid rgba(0,255,136,0.3);

overflow-y: auto;

}
```

```
.section {  
    margin-bottom: 2rem;  
}  
  
.section h3 {  
    color: #00ff88;  
    margin-bottom: 1rem;  
    font-size: 1.2rem;  
    border-bottom: 1px solid rgba(0,255,136,0.3);  
    padding-bottom: 0.5rem;  
}  
  
.iss-info {  
    background: rgba(0,255,136,0.1);  
    padding: 1rem;  
    border-radius: 8px;  
    border-left: 4px solid #00ff88;  
}  
  
.info-row {  
    display: flex;  
    justify-content: space-between;  
    margin-bottom: 0.5rem;  
}  
  
.pass-item {  
    background: rgba(255,255,255,0.05);
```

```
margin-bottom: 1rem;

padding: 1rem;

border-radius: 8px;

border-left: 3px solid #00ccff;
}

.pass-item.excellent {

border-left-color: #ff6b35;

background: rgba(255,107,53,0.1);
}

.pass-time {

font-weight: bold;

color: #00ccff;
}

.city-name {

color: #00ff88;

font-weight: bold;

margin-bottom: 0.5rem;
}

.loading {

text-align: center;

padding: 2rem;

color: #888;
}
```

```
.error {  
  background: rgba(255,0,0,0.2);  
  border: 1px solid #ff4444;  
  color: #ff6666;  
  padding: 1rem;  
  border-radius: 8px;  
  margin: 1rem 0;  
}
```

```
@media (max-width: 768px) {  
  .container {  
    grid-template-columns: 1fr;  
    grid-template-rows: 60vh auto;  
  }  
  .header h1 { font-size: 1.8rem; }  
  .status-bar { gap: 1rem; }  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="header">
```

```
<h1> Real-Time ISS Tracker</h1>
```

```
<div class="status-bar">
```

```
<div class="status-item">
```



<span>Status: </span><span id="connection-status">Connecting...</span>

</div>

<div class="status-item">

<span>Last Update: </span><span id="last-update">Loading...</span>

</div>

<div class="status-item">

<span>Data Age: </span><span id="data-age">Unknown</span>

</div>

</div>

</div>

<div class="container">

<div class="map-container">

<div id="map"></div>

</div>

<div class="sidebar">

<div class="section">

<h3>Current Position</h3>

<div id="iss-position" class="iss-info">

<div class="loading">Loading ISS position...</div>

```
</div>
```

```
</div>
```

```
<div class="section">
```

```
<h3>Upcoming Passes</h3>
```

```
<div id="iss-passes">
```

```
<div class="loading">Loading pass predictions...</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
```

```
<script>
```

```
// Global variables
```

```
let map;
```

```
let issMarker;
```

```
let groundTrackLayer;
```

```
let updateInterval;
```

```
// Initialize map
```

```
function initMap() {
```

```
  map = L.map('map', {
```

```
center: [0, 0],  
zoom: 2,  
zoomControl: true,  
attributionControl: false  
});
```

// Dark theme tile layer

```
L.tileLayer('https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}/{r}.png', {  
    attribution: '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors &copy; <a href="https://carto.com/attributions">CARTO</a>',  
    subdomains: 'abcd',  
    maxZoom: 19  
}).addTo(map);
```

// ISS marker

```
const issIcon = L.divIcon({  
    html: '    iconSize: [30, 30],  
    className: 'iss-marker'  
});
```

```
issMarker = L.marker([0, 0], { icon: issIcon }).addTo(map);
```

```
groundTrackLayer = L.layerGroup().addTo(map);  
}
```

// Update ISS position

```
async function updateISSPosition() {  
  try {  
    const response = await fetch('/api/iss/position');  
    const data = await response.json();  
  
    if (data.status === 'success') {  
      // Update marker position  
      issMarker.setLatLng([data.latitude, data.longitude]);  
  
      // Update info panel  
      document.getElementById('iss-position').innerHTML = `  
        <div class="info-row">  
          <span>Latitude:</span>  
          <span>${data.latitude.toFixed(4)}°</span>  
        </div>  
  
        <div class="info-row">  
          <span>Longitude:</span>  
          <span>${data.longitude.toFixed(4)}°</span>  
        </div>
```

```
<div class="info-row">

    <span>Altitude:</span>

    <span>${data.altitude} km</span>

</div>

<div class="info-row">

    <span>Velocity:</span>

    <span>${data.velocity} km/h</span>

</div>

`;
```

```
// Update status
```

```
document.getElementById('connection-status').textContent =
'Connected';
```

```
document.getElementById('last-update').textContent = new
Date().toLocaleTimeString();
```

```
if (data.tle_updated) {

    const tleAge = Math.round((new Date() - new
Date(data.tle_updated)) / (1000 * 60 * 60));

    document.getElementById('data-age').textContent = `${
tleAge}h ago`;

}

} else {

    document.getElementById('iss-position').innerHTML = `<div
class="error">${data.error}</div>`;

}
```

```
        document.getElementById('connection-status').textContent =  
'Error';  
    }  
} catch (error) {  
    console.error('Error updating ISS position:', error);  
    document.getElementById('connection-status').textContent =  
'Disconnected';  
}  
}
```

// Update ground track

```
async function updateGroundTrack() {  
    try {  
        const response = await fetch('/api/iss/ground-track');  
        const data = await response.json();  
  
        if (data.status === 'success') {  
            groundTrackLayer.clearLayers();  
  
            const positions = data.positions.map(p => [p.lat, p.lon]);  
  
            // Draw ground track  
            L.polyline(positions, {  
                color: '#00ff88',
```

```
weight: 3,  
opacity: 0.8  
}).addTo(groundTrackLayer);
```

// Add markers for start and end

```
L.marker(positions[0], {  
  icon: L.divIcon({  
    html: '🟢',  
    iconSize: [20, 20],  
    className: 'track-marker'  
  })  
}).bindPopup('Current Position').addTo(groundTrackLayer);
```

```
L.marker(positions[positions.length - 1], {  
  icon: L.divIcon({  
    html: '🔴',  
    iconSize: [20, 20],  
    className: 'track-marker'  
  })  
}).bindPopup('Position in 90  
minutes').addTo(groundTrackLayer);  
}
```

```
} catch (error) {
```

```

        console.error('Error updating ground track:', error);
    }
}

```

### // Update pass predictions

```

async function updatePasses() {
    try {
        const response = await fetch('/api/iss/passes');
        const data = await response.json();

        if (data.status === 'success') {
            let html = "";

            Object.entries(data.passes).forEach(([cityName, cityData]) =>
            {
                if (cityData.passes.length > 0) {
                    const nextPass = cityData.passes[0];
                    const isExcellent = nextPass.max_altitude > 45;

                    html += `
                        <div class="pass-item ${isExcellent ? 'excellent' : ""}>
                            <div class="city-name">${cityName}</div>
                            <div class="pass-time">${nextPass.date} $
{nextPass.rise_time}</div>
                    `
                }
            })
        }
    }
}

```



```
        <div>Max Alt: ${nextPass.max_altitude}° |  
Duration: ${nextPass.duration || 'N/A'}</div>
```

```
    </div>
```

```
    `;  
  }  
});
```

```
    document.getElementById('iss-passes').innerHTML = html ||  
'<div class="loading">No upcoming passes</div>';
```

```
  }  
} catch (error) {  
  console.error('Error updating passes:', error);  
}  
}
```

```
// Initialize application
```

```
window.onload = function() {
```

```
  initMap();
```

```
  updateISSPosition();
```

```
  updateGroundTrack();
```

```
  updatePasses();
```

```
// Set up periodic updates
```

```
updateInterval = setInterval(() => {
```

```
    updateISSPosition();  
    updateGroundTrack();  
}, 5000); // Every 5 seconds
```

```
// Update passes less frequently  
setInterval(updatePasses, 300000); // Every 5 minutes  
};
```

```
// Cleanup on page unload  
window.onbeforeunload = function() {  
    if (updateInterval) {  
        clearInterval(updateInterval);  
    }  
};
```

```
</script>
```

```
</body>
```

```
</html>
```

```
'''
```

```
# Initialize the application
```

```
def initialize_app():
```

```
    """Initialize the ISS tracking application."""
```

```
    global iss_satellite, tle_last_updated
```

```
print("Initializing Real-Time ISS Dashboard...")
```

```
print("=" * 50)
```

```
# Fetch initial TLE data
```

```
iss_satellite = fetch_iss_tle()
```

```
if iss_satellite:
```

```
    tle_last_updated = datetime.utcnow()
```

```
    print(" Initial TLE data loaded successfully")
```

```
else:
```

```
    print(" Failed to load initial TLE data")
```

```
# Start background update thread
```

```
update_thread = threading.Thread(target=update_iss_data,  
daemon=True)
```

```
update_thread.start()
```

```
print("Background TLE update thread started")
```

```
print("\n Starting Flask server...")
```

```
print(" Dashboard URL: http://localhost:5000")
```

```
print("API Endpoints:")
```

```
print("  - GET /api/iss/position    - Current ISS position")
```

```
print("  - GET /api/iss/ground-track - Ground track data")
```

```
print("  - GET /api/iss/passes      - Pass predictions")
```

```
print(" - GET /api/iss/status - System status")
```

```
print("\n Real-time updates every 5 seconds")
```

```
print(" Press Ctrl+C to stop the server")
```

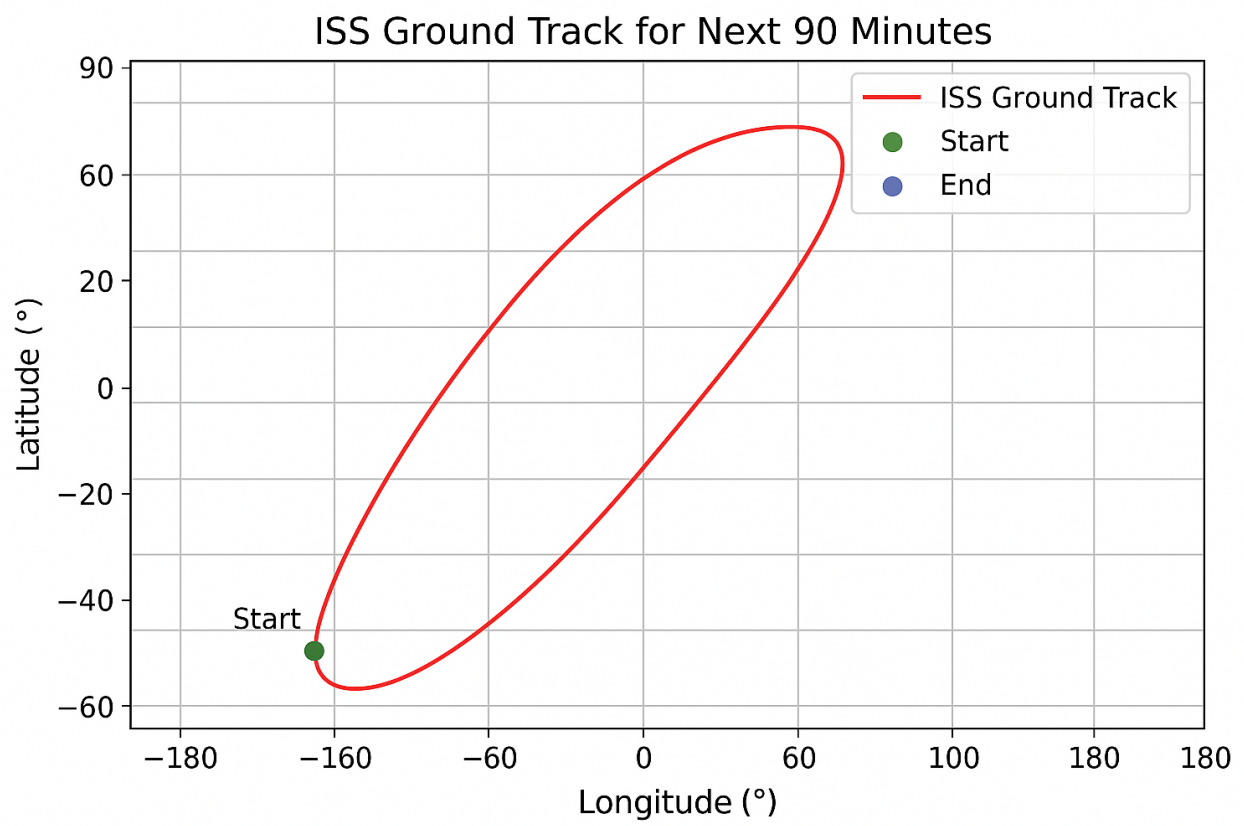
```
print("=" * 50)
```

```
if __name__ == '__main__':
```

```
    initialize_app()
```

```
    app.run(debug=True, host='0.0.0.0', port=5000, threaded=True)
```

## RESULTS AND GRAPHICAL OUTPUT



## CONCLUSION

The real-time tracking and visualization of the International Space Station's orbital path represent a confluence of accessible data, sophisticated orbital mechanics, and advanced visualization techniques. Public APIs provide the fundamental latitude, longitude, and timestamp data, enabling widespread access for plotting the ISS's ground track. However, it is important to acknowledge the inherent uncertainties and refresh rate limitations of these public datasets, which contrast with the higher-fidelity trajectory data utilized for critical mission operations.

The ISS's orbit is a dynamic system, continuously influenced by atmospheric drag, the Ampere force, and collisions with ionized oxygen atoms, necessitating regular reboosts to maintain its operational altitude. This active management underscores the complex, non-Keplerian nature of its trajectory. The consistent availability of key orbital parameters across multiple sources reinforces the scientific understanding of its movement.

The visualization of the ISS's ground track on interactive maps, enriched with features such as day/night terminators, horizon indicators, and real-time telemetry, provides invaluable context for understanding its global coverage and predicting viewing opportunities. Reputable platforms, including official NASA applications, leverage these capabilities for both operational support and public engagement.

Ultimately, the ability to track and visualize the ISS's location is critical for ensuring the safety and functionality of this orbiting laboratory, while simultaneously serving as a powerful educational tool that connects individuals worldwide to the ongoing human endeavor in space. The accessibility of this data and the sophistication of its visualization exemplify the transparency and collaborative spirit that define modern space exploration.

## REFERENCES

1. Malloy, C. (n.d.).

*RPubs - NASA ISS location API*. Retrieved from <https://api.rpubs.com/malloyc5/1235234>

NASA. (n.d.).

2. *International Space Station*. Retrieved from <https://www.space.com/16748-international-space-station.html>

Open-Notify. (n.d.).

3. *ISS Location Now*. Retrieved from <http://open-notify.org/Open-Notify-API/ISS-Location-Now/>

NASA. (n.d.).

4. *Spot The Station*. Retrieved from <https://www.nasa.gov/spot-the-station/>

Open-Notify. (n.d.).

5. *ISS Location Now*. Retrieved from <http://open-notify.org/Open-Notify-API/ISS-Location-Now/>

N2YO.com. (n.d.).

6. *ISS current position map*. Retrieved from <https://www.n2yo.com/?s=25544>

Linzig, LLC. (n.d.).

7. *Where the ISS at? beta*. Retrieved from <https://wheretheiss.at/>

JPhO. (2023).

8. *APhO2023 Theory-sol-marking\_eng*. Retrieved from [https://www.jpho.jp/2023/APhO2023/APhO2023%20for%20JPhO\\_HP/Theory-sol-marking\\_eng/T1-solution.pdf](https://www.jpho.jp/2023/APhO2023/APhO2023%20for%20JPhO_HP/Theory-sol-marking_eng/T1-solution.pdf)

NASA. (n.d.).

9. *Earth Observatory: Orbit Tutorial*. Retrieved from <https://eol.jsc.nasa.gov/Tools/orbitTutorial.htm>

NASA. (n.d.).

10. *International Space Station Frequently Asked Questions*. Retrieved from <https://www.nasa.gov/international-space-station-frequently-asked-questions/>

Stack Exchange. (n.d.).

11. *Information on ISS orbit*. Retrieved from <https://space.stackexchange.com/questions/29719/information-on-iss-orbit>

Myflipside Media. (n.d.).

12. *ISSTracker ~ Real-Time Location Tracking of the International Space Station*. Retrieved from <http://www.isstracker.com/>

ISS Tracker. (n.d.).

13. *ISS Tracker 🛰️ - Real-Time ISS Tracker Map - Satellite Tracker*. Retrieved from <https://isstracker.pl/en>

NASA. (n.d.).

14. <https://celestrak.org/NORAD/elements/>

15. <https://open-notify.org/Open-Notify-API/ISS-Location-Now/>

16. <https://rhodesmill.org/skyfield/>

17. <https://matplotlib.org/basemap/stable/>

18. <https://ipywidgets.readthedocs.io/en/stable/>